

# Übungsblatt 7: Performance & Pipelining

## Mikrocomputertechnik 1

### Zielsetzung

In diesem Übungsblatt verlassen wir den ineffizienten Single-Cycle-Prozessor und aktivieren die Fließbandarbeit (Pipelining). Sie stellen den Ripes-Simulator auf das 5-Stage-Modell um und untersuchen, wie die Hardware mit Kollisionen (Hazards) umgeht. Sie lösen Data Hazards durch Forwarding, beobachten Pipeline-Bubbles (NOPs) bei Load-Use-Konflikten und analysieren fehlerhafte Instruktionen bei Sprüngen (Flushing).

### Aufgabe 1: Latenz vs. Durchsatz

Das Pipelining ist eine Architektur-Revolution, unterliegt aber physikalischen Gesetzen. Beantworten Sie folgende Fragen.

#### Latenz eines einzelnen Befehls

Verkürzt die Einführung einer 5-stufigen Pipeline die Ausführungszeit (Latenz) eines einzelnen, isolierten Befehls (z. B. von Anfang IF bis Ende WB)? Was genau wird durch das Fließband gesteigert?

#### Idealer Speedup

Wie hoch ist der theoretisch ideale Speedup (Leistungsgewinn) einer 5-stufigen Pipeline im Vergleich zum Single-Cycle-Prozessor? Nennen Sie einen Grund (Overhead/Imbalance), warum dieser ideale Faktor in der Realität nie exakt erreicht wird.

### Ihre Lösung

*(Notieren Sie Ihre Antworten hier.)*

## Aufgabe 2: Data Hazard und Forwarding (Ripes)

Stellen Sie in Ripes oben links zwingend den **5-Stage Processor** ein.

Geben Sie folgenden Code ein:

```
.text
li t0, 50
li t1, 100
add t2, t0, t1    # schreibt t2
sub a0, t2, t0     # liest t2 sofort danach
```

Hinweis: `li` ist ein Pseudo-Befehl und expandiert in Ripes oft zu mehreren Maschinenbefehlen. Zählen Sie im Pipeline-Tab die echten Instruktionen.

### RAW Data Hazard

Warum entsteht zwischen dem `add`- und dem `sub`-Befehl ein Read-After-Write-(RAW)-Data-Hazard auf dem Fließband? In welchen Pipeline-Stufen befinden sich die beiden Befehle typischerweise, wenn `sub` in der Decode-Stufe `t2` lesen will?

### Forwarding in Ripes

Ticken Sie in Ripes mit Clock vorwärts, bis sich der `sub`-Befehl in der EX-Stufe befindet. Schauen Sie in den Processor-Tab: Aus welchem Pipeline-Register greift der Multiplexer das Forwarding-(Bypass)-Signal ab, um den Hazard aufzulösen, ohne das Fließband anzuhalten?

### Ihre Lösung

*(Notieren Sie Ihre Beobachtungen hier.)*

## Aufgabe 3: Load-Use-Stalls und Compiler-Scheduling

Löschen Sie den Code und provozieren Sie die physikalische Grenze des Forwardings:

```

.data
array: .word 42

.text
la s1, array
lw t0, 0(s1)
add a0, t0, t0

```

## Stall und Bubble

Ticken Sie in Ripes vorwärts. Wechseln Sie in den Tab Pipeline (Zeit-Takt-Diagramm).

1. Was passiert zwischen dem `lw`- und dem `add`-Befehl visuell (Bubble/Stall)?
2. Warum konnte die Hardware diesen Konflikt nicht per Bypass-Kabel lösen?

## Instruction Scheduling

Werden Sie zum Compiler: Fügen Sie einen völlig unabhängigen Befehl (z. B. `addi a5, x0, 1`) exakt zwischen `lw` und `add` ein. Beobachten Sie erneut das Pipeline-Tab. Was hat sich durch dieses Instruction Scheduling verändert?

## Ihre Lösung

*(Notieren Sie Ihre Beobachtungen hier.)*

## Aufgabe 4: Control Hazards und Flushing

Bauen Sie eine kurze Endlosschleife, um Control Hazards (Sprünge) zu untersuchen:

```

.text
li t0, 10
Loop:
addi t0, t0, -1
bne t0, zero, Loop

```

## Flushing bei Branch Taken

Ticken Sie mit der Clock, bis der `bne`-Befehl seine Entscheidung trifft (Branch Taken) und der Program Counter zurück zu `Loop` springt.

Betrachten Sie das Pipeline-Tab:

1. Was macht die CPU mit den ein oder zwei Befehlen, die sie blind in den vorderen Stufen (IF und/oder ID) hinter der Schleife geladen hatte?
2. Wie nennt man diesen Vorgang?

## Ihre Lösung

*(Notieren Sie Ihre Beobachtungen hier.)*

## Aufgabe 5: Pipeline-Raster selbst ausfüllen

Gegeben sei die folgende Befehlsfolge auf einer 5-stufigen Pipeline **mit Forwarding und Hazard Detection** (IF, ID, EX, MEM, WB). Tragen Sie die Stufen ein und markieren Sie Stalls klar (z. B. STALL oder NOP).

```
lw    t0, 0(a0)
addi  t1, t0, 1
add   t2, t1, t0
```

| Befehl \ Takt  | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|----------------|---|---|---|---|---|---|---|---|
| lw t0, 0(a0)   |   |   |   |   |   |   |   |   |
| addi t1, t0, 1 |   |   |   |   |   |   |   |   |
| add t2, t1, t0 |   |   |   |   |   |   |   |   |

## Ihre Lösung

*(Füllen Sie das Raster aus.)*